

# Паттерны эффективной работы с Record и RecordSet в Python

Платформа СБИС написана на языке C++. При работе с типами Платформы на Python происходит межъязыковое взаимодействие, которое вносит накладной расход на преобразование типов. В зависимости от того, как организовать работу, этот накладной расход может как увеличиваться, так и уменьшаться.

В качестве примера рассмотрим работу с полем на Python.

```
1 rec=GetSomeRecord()  
2 x=rec.typeDoc
```

В этом примере "за кадром" происходит следующее:

1. Построение C++ строки по Python-строке.
2. Обращение к полю и извлечение из него значения.
3. Разрушение временной C++ строки.
4. Далее полученное значение транслируется в объект числового типа Python'a.

Как видно из примера, даже в ходе такой простой операции присутствуют ощутимые накладные расходы.

## Примеры паттернов

Рассмотрим примеры повышения эффективной работы с данными.

### Создание RecordSet из данных в Python

Метод [CreateRecordSet](#) рекурсивно создает вложенные RecordSet и Record. Для этого при добавлении поля в формат нужно указать вложенный формат и передать либо данные для создания, аналогичные аргументу data, либо уже готовый RecordSet или Record.

Если передан list[list], то данные будут сопоставляться по номеру полей, т.е. в первое поле из формата запишется первое поле из list:

```
CreateRecordSet(frmt, data: list[list])
```

Если передан list[dict], то сопоставление выполняется по имени полей:

```
CreateRecordSet(frmt, data: list[dict])
```

Аналогично создается один Record из набора данных:

```
1 Record(frmt, data: list)  
2 Record(frmt, data: dict)
```

### Работа с dataclass'ами и нативными типами

Если мы пишем некий расчетный код, и наша цель не получить RecordSet, а что-то посчитать, мы можем использовать набор dataclass'ов, описывающих нашу информационную модель. Для работы с dataclass'ами используется метод [RecordSet.ToListOf](#):

```
RecordSet.ToListOf(OurDataClass, fields: str | list[str]) ->
```

```
list[OurDataClass]
```

Метод аналогичен [RecordSet.ToList](#), но для каждой строки вызывается конструктор dataclass, в который передаются значения полей в fields, в том же порядке, согласно списку.

Первый аргумент метода - любой Callable, который принимает список значений полей и что-то возвращает, т.е. можно передать фабричный метод.

## Работа с RecordSet не требуется

Если получение RecordSet'a не нужно, используйте метод [SqlQueryOf](#):

```
SqlQueryOf(OurDataClass, <стандартные аргументы SqlQuery>) ->  
list[OurDataClass]
```

Метод работает аналогично RecordSet.ToListOf, но поля передать нельзя, т.к. они все необходимы.

## Оптимизация работы со списочным методом

Допустим, у нас есть списочный метод и нужно соблюдать списочный контракт. Однако часть возвращаемых данных мы получаем из запроса к БД, а остальные – из других источников.

В этом случае оптимально исключить добавление колонок к готовому набору данных.

Все требуемые колонки передаются в метод выполнения запроса с помощью формата ожидаемого результата.

Колонки, полученные из БД - заполняются данными из БД, а те, которых нет в запросе, но необходимы – будут иметь значение NULL.

Это быстрее, чем использование метода [Migrate](#), т.к. метод перекладывает данные со старого формата на новый.

## Быстрое объединение данных из разных источников в один RecordSet

Используйте класс [Query](#) для объединения RecordSet'ов. Класс позволяет соединить несколько RecordSet'ов за один вызов.

Это возможно даже в случае, если есть какие-то расчетные данные в Python и их нужно добавить в колонки RecordSet. Можно создать из них второй RecordSet и соединить его с первым через Query.

## Типовые примеры неэффективной работы с записями

Для снижения накладных расходов и повышения производительности необходимо придерживаться двух стратегий:

- Не делать лишнего.
- Укрупнять операции.

Рассмотрим типовые случаи неэффективной работы с записями.

### Добавление Record в RecordSet

Приведенный ниже пример неэффективен. В этом примере происходит следующее:

1. Создаётся объект res по формату [RecordSet](#)'а.
2. Каждое поле заполняется по-отдельности.
3. С заполненного [Record](#)'а снимается копия и помещается в RecordSet.

```
1 # Плохо  
2  
3 # Создаём экземпляр класса sbis.RecordSet  
4 rs = sbis.RecordSet(format)  
5  
6 # Создание записи по формату RecordSet  
7 rec = sbis.Record(format)  
8  
9 """ Результатом вызова оператора [ ] является объект IField,
```

```

10  который умирает после метода From
11  """
12  rec['@Phonebook'].From(1)
13  rec['FIO'].From('Сидоров С.С.')
14  rec['Age'].From(30)
15  rec['Human'].From(False)
16
17  # Копирование записи в RecordSet
18  rs.AddRow(rec)

```

Так как объект `rec` является полностью независимым и далее в программе может быть изменен каким-либо образом (в том числе со сменой формата), `RecordSet` вынужден снимать копию с добавляемой в него записи, чтобы избежать нарушения своих инвариантов.

В этом примере происходит создание `Record`'а сразу внутри `RecordSet`'а, поэтому нет необходимости создавать отдельный объект. Однако заполнение `Record`'а всё еще нельзя считать эффективным.

```

1  # Хорошо
2
3  # Создаётся внутренняя ссылка на объект типа RecordSet
4  rec = rs.AddRow()
5
6  # Заполнение записи по формату RecordSet'a
7  rec.Set('@Phonebook', 2)
8  rec.Set('FIO', 'Сидоров С.С.')
9  rec.Set('Age', 30)
10 rec.Set('Human', False)

```

Заполнение множества поле `RecordSet`'а также неэффективно. Об этом подробно описано ниже.

## Заполнение множества полей `Record`

В данном примере происходит заполнение полей `Record`'а, при этом каждое поле заполняется по отдельности.

При заполнении поля вызывается оператор `[]`, что приводит к созданию временного объекта [IField](#), который разрушается после метода [From](#). Такой способ считается неэффективным, так как требует множественного обращения к полям записи, и создания для каждого поля своего временного объекта.

В текущей реализации создаются и разрушаются исключительно C++-базированные инстансы, которые служат для обращения к ним из Python через мост. Однако в случае выполнения простых операций чтения и записи данных этот способ является избыточным.

```

1  # Плохо
2
3  rec = GetSomeRecord()
4
5  """ Результатом вызова оператора [] является объект IField,
6  который умирает после метода From
7  """
8  rec["Foo"].From(10)
9  rec["Bar"].From(True)
10 rec["Baz"].From('hello world')

```

В данном примере происходит заполнение множества полей `Record`'а с помощью метода [Fill](#). Данный метод является эффективным, так как формирует строку записи по словарю переданных значений, а, следовательно, не требует создания для каждого поля отдельных временных объектов.

```

1  # Хорошо
2
3  rec = GetSomeRecord()
4
5  # метод Fill заполняет поля записи значениями из переданного словаря
6  rec.Fill({"Foo": 10, "Bar": True, "Baz": 'hello world'})

```

## Проверка наличия полей

В данном примере происходит поиск поля с именем "Foo" в RecordSet'e путем перебора записей в цикле по условию. Этот способ является неэффективным. RecordSet является таблицей. Следовательно, если в формате RecordSet'a существует поле, имеющее искомое имя, значит оно есть во всех записях.

Рассмотрим алгоритм поиска поля с именем "Foo". Метод [TestField](#) возвращает объект [TestFieldResult](#), если поле с искомым именем имеется в формате записи. Данный метод используется в цикле, в результате чего для каждой записи формируется свой объект TestFieldResult.

```
1 # Плохо
2
3 result_rs = CreateResultRS()
4 helper_rs = GetHelperRS()
5 for idx, rec in enumerate(helper_rs):
6     result_rec = result_rs[idx]
7     result_rec.Set("Bar", rec["Bar"]) # поле Bar обязано быть и его
наличие не проверяем
8     if rec.TestField("Foo") is not None:
9         result_rec["Foo"].From(rec["Foo"])
```

Обратимся к более эффективному примеру.

В данном примере наличие поля проверяется однократно по формату RecordSet'a. Метод [Format](#) возвращает формат RecordSet'a, он имеет тип [RecordFormat](#).

Наличие поля с искомым значением проверяется с помощью специального метода [\\_\\_contains\\_\\_](#). Если поле со значением "Foo" найдено в формате RecordSet'a, то выражение истинно. Если поле не найдено – выражение становится ложным.

Таким образом, в цикле не происходит непосредственного обращения к полям записи, для проверки условия используется значение переменной helper\_rs\_has\_foo.

```
1 # Хорошо
2
3 result_rs = CreateResultRS()
4 helper_rs = GetHelperRS()
5 helper_rs_has_foo = "Foo" in helper_rs.Format()
6 for idx, rec in enumerate(helper_rs):
7     result_rec = result_rs[idx]
8     result_rec.Set("Bar", rec["Bar"]) # поле Bar обязано быть и его
наличие не проверяем
9     if helper_rs_has_foo:
10         result_rec.Set("Foo", rec["Foo"])
```

## "Ложная" оптимизация

Проверка установленного значения в записи при установке такого же значения не требуется.